Instituto Tecnológico de Costa Rica

Área Académica de Ingeniería en Computadores

Paradigmas de Programación (CE1106)

Tarea1

Profesor:

Marco Rivera Meneses

Estudiantes:

Jestim Espinoza Morales – 2023282666

Primer Semestre 2026

---

1. Descripción detallada del algoritmo de solución

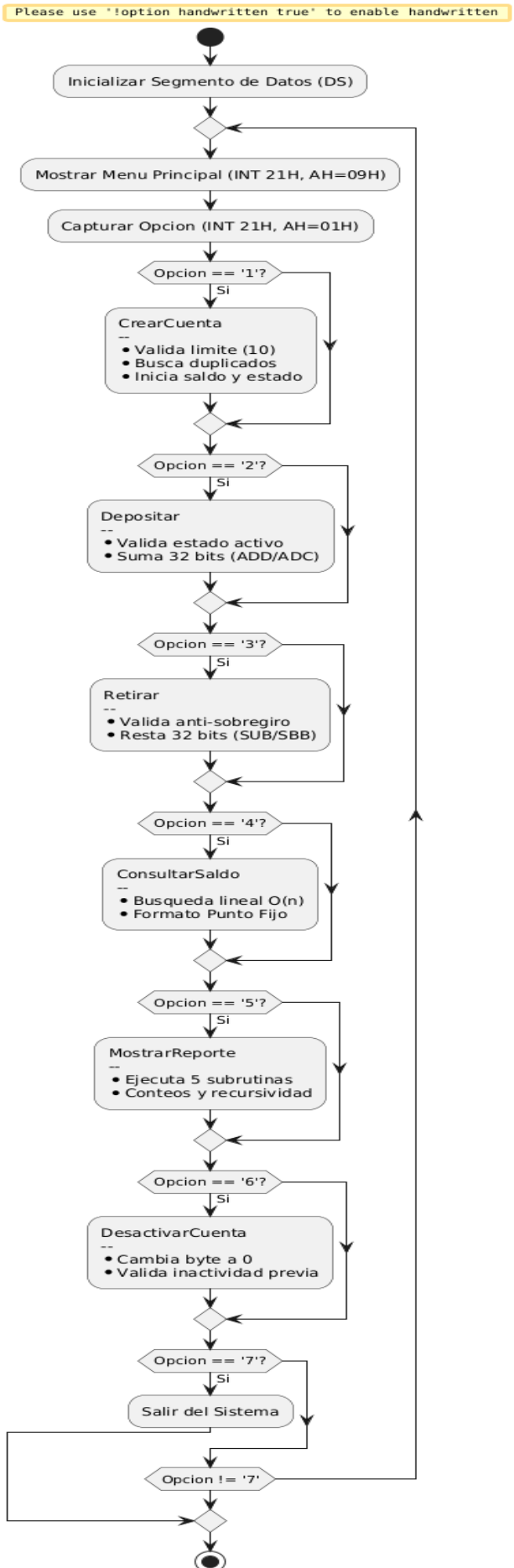
El sistema BankTec fue diseñado bajo el paradigma de programación imperativa y estructurada. El algoritmo central opera mediante un bucle infinito controlado por eventos (`MenuPrincipal`), el cual interrumpe su ejecución únicamente cuando el usuario selecciona explícitamente la opción de salida.

El flujo de datos se basa en la manipulación directa de la memoria mediante direccionamiento indexado con base (`[BX+SI+offset]`). En lugar de utilizar bases de datos dinámicas, el algoritmo reserva estáticamente un bloque de 280 bytes en el segmento de datos (`.DATA`), dividiéndolo lógicamente en 10 bloques de 28 bytes. Las operaciones de búsqueda (necesarias para depositar, retirar o validar duplicados) emplean un algoritmo de búsqueda lineal `O(n)`, iterando secuencialmente sobre las estructuras activas utilizando la instrucción `LOOP` vinculada al registro contador `CX`.

Uno de los algoritmos más complejos de esta solución es la implementación de aritmética de 32 bits en un entorno de 16 bits. Para evitar el desbordamiento (`overflow`) en saldos bancarios grandes, el algoritmo divide lógicamente los montos en dos palabras de memoria contiguas (Parte Baja y Parte Alta). Las sumas y restas se ejecutan en cascada: primero se operan las partes bajas (`ADD` / `SUB`), y luego las partes altas incorporando la bandera de acarreo (Carry Flag) mediante las instrucciones `ADC` (Add with Carry) y `SBB` (Subtract with Borrow).

Adicionalmente, el algoritmo resuelve la carencia de una Unidad de Punto Flotante (FPU) implementando aritmética de Punto Fijo. Todos los valores ingresados por el usuario se escalan internamente multiplicándolos por 10,000. Esto permite procesar hasta cuatro cifras decimales con la precisión y velocidad de la aritmética entera, formateando la salida visualmente mediante la inserción del carácter `.` en la posición correcta durante la fase de impresión.

Algoritmo de Solucion: Flujo Principal BankTec



2. Descripción de los procedimientos/funciones implementadas

El código fuente ha sido modularizado, asegurando que cada funcionalidad del negocio tenga un procedimiento asilado (`PROC`). A continuación, se detallan:

- **MenuPrincipal**: Actúa como el controlador principal (`dispatcher`). Despliega el texto de la interfaz gráfica en consola utilizando la interrupción del DOS `INT 21H, AH=09H`, captura la decisión del usuario mediante `AH=01H` y redirige el flujo de control hacia el procedimiento correspondiente usando una estructura de decisión basada en saltos condicionales (`CMP` seguido de `JE`).
- **CrearCuenta**: Procedimiento de registro y asignación de memoria. Primero, valida que la variable `num_cuentas` sea menor a 10. Luego, calcula el offset base multiplicando el índice actual por 28 (`TAM_CUENTA`). Captura la cadena de texto del nombre (usando `AH=0Ah`) y el número de cuenta. Llama a la subrutina auxiliar de búsqueda para asegurar que el número de cuenta sea único. Finalmente, inicializa el saldo y establece el byte de estado forzosamente en `1` (Activa).
- **BuscarCuentaPorNumero**: Recibe en el registro `BX` el número de cuenta objetivo y recorre el arreglo `cuentas` en la memoria. Si encuentra una coincidencia exacta, retorna en el registro `AX` la posición relativa en bytes (el offset almacenado en `SI`). Si agota el ciclo iterativo sin éxito, retorna el valor hexadecimal `0FFFFh` en `AX` actuando como bandera para indicar un error de "elemento no encontrado".
- **Depositar**: Procedimiento transaccional de adición. Reutiliza la función de búsqueda para localizar el bloque de memoria de la cuenta. Inmediatamente después, valida la regla de negocio principal accediendo al byte de `ESTADO`; si la cuenta está inactiva, aborta y bloquea la transacción. Si es válida, solicita el monto, verifica que sea mayor a cero y ejecuta la matemática de 32 bits (`ADD` a la baja, `ADC` a la alta) guardando el resultado directo en memoria.
- **Retirar**: Procedimiento transaccional. Comparte las validaciones iniciales preventivas de `Depositar` (existencia y estado activo). Validación de fondos ("anti-sobregiro"): ejecuta una comparación relacional de 32 bits confrontando primero las palabras altas y, únicamente en caso de empate, las palabras bajas. Si el saldo en memoria es superior o igual al monto solicitado, procede con la deducción utilizando las instrucciones `SUB` y `SBB`.
- **EncontrarMayorSaldo y EncontrarMenorSaldo**: Funciones de análisis de datos que implementan un patrón de recursividad de cola (*tail recursion*). Emplean activamente la pila del sistema (`STACK`) mediante `PUSH` y `POP` para preservar intacto el estado de los registros locales en cada llamada recursiva anidada. En el caso base, asumen la primera cuenta evaluada como el extremo y, en la fase de retorno, efectúan comparaciones secuenciales de 32 bits actualizando variables globales (`mayor_saldo` / `menor_saldo`) si hallan una magnitud que rompa el límite previo.

- **MostrarReporte**: Invoca secuencialmente diversas subrutinas específicas que recorren la estructura de datos. Suma iterativamente las cuentas activas e inactivas y extrae el gran total acumulando variables.
- **StringToWord y WordToString**: Funciones responsables de la traducción entre representación humana y máquina. `StringToWord` lee caracteres ASCII desde un búfer en memoria, les resta `30h` para obtener su valor numérico real, y los integra a una variable acumuladora multiplicando progresivamente por la base del sistema (`10`). `WordToString` efectúa la operación inversa, aplicando el teorema de división repetida entre 10 y guardando sistemáticamente los residuos resultantes.
- **LeerMonto y ValidarMonto**: `ValidarMonto` actúa como un analizador léxico, iterando sobre la cadena ingresada para asegurar que solo contenga caracteres comprendidos entre `'0'` y `'9'`, el separador decimal `'.'`, y rechazando categóricamente el signo `'-'` u otras letras para prevenir la corrupción de datos y la inyección de saldos negativos.

3. Descripción de la ejemplificación de las estructuras de datos desarrolladas

Se diseñó una estructura contigua tipo "Registro" (Struct C-like) de 28 bytes de longitud. Se accede de forma algorítmica mediante el direccionamiento Indexado con Base (`[BX + SI + OFFSET]`).

- **Offset 0** (`NUM_CUENTA`, 2 bytes): Maneja números de identificación en un rango de 0 a 65,535 (WORD).
- **Offset 2** (`NOMBRE`, 21 bytes): Permite guardar el identificador alfanumérico del cliente (20 caracteres efectivos más el carácter terminador `$` exigido por la consola DOS).
- **Offset 23** (`SALDO`, 4 bytes): Compuesto algorítmicamente por dos variables adyacentes de 16 bits (WORD) que se tratan lógicamente en las rutinas matemáticas como un `DWORD`. Es capaz de retener montos transaccionales masivos emulando 4 decimales.
- **Offset 27** (`ESTADO`, 1 byte): Actúa como una bandera booleana a nivel binario. El valor `1` habilita la cuenta para operar, y el valor `0` la congela a nivel transaccional. El sistema reserva un búfer continuo de 280 bytes (`10 * 28 bytes = 280 DUP`) en tiempo de compilación.

4. Problemas Encontrados y Soluciones

- **Manejo de decimales en arquitectura 8086**: El procesador base carece nativamente de una Unidad de Punto Flotante (FPU o coprocesador 8087), impidiendo usar variables `float`.
    - *Solución técnica implementada*: Se diseñó un algoritmo de "Punto Fijo" (Fixed-Point Arithmetic). El programa multiplica internamente los montos ingresados por procesándolos como enteros completos de 32 bits, y a la hora de imprimirlos,

inserta un separador `.` visual cuatro posiciones antes del final de la cadena ASCII.

- **Pérdida de valores posicionales al imprimir enteros**: Al dividir números entre 10 para imprimirlos caracter por caracter, los dígitos se generaban algorítmicamente en orden inverso (ej. el número 150 generaba '0', '5', '1').
    - *Solución técnica implementada*: Se utilizó de forma estricta la estructura de datos Pila (`Stack`) del sistema. Mediante las instrucciones `PUSH` se almacenó temporalmente cada dígito calculado y luego con `POP` se extrajeron empleando el esquema LIFO (Last In, First Out), restaurando y garantizando así el orden posicional natural de izquierda a derecha.

5. Problemas conocidos sin solución

No hubo problemas sin solución. Todas las funcionalidades técnicas exigidas, reglas del negocio bancario y requerimientos del sistema especificados detalladamente en el documento de consigna (TareaASMI2026.pdf) fueron implementados exitosamente y demostrados funcionales.

6. Plan de Actividades

1. **Definir Estructuras**: Crear estructura en memoria para saldo (4 bytes), nombre (21 bytes), estado (1 byte) y número (2 bytes). *Responsable: Jestim Espinoza. Tiempo: 1 día.*
2. **CrearCuenta**: Desarrollar la función de registro validando duplicados y saldos iniciales >= 0. *Responsable: Jestim Espinoza. Tiempo: 2 días.*
3. **Depósitos y Retiros**: Programar funciones matemáticas de 32 bits para modificar saldos, validando cuentas activas y evitando sobregiros. *Responsable: Jestim Espinoza. Tiempo: 2 días.*
4. **Consultas y Desactivación**: Desarrollar las subrutinas de consulta con impresión formateada y alteración segura del byte de estado. *Responsable: Jestim Espinoza. Tiempo: 1 día.*
5. **Reportes**: Implementar las funciones de conteo, suma total del banco y algoritmos recursivos de búsqueda del mayor/menor saldo. *Responsable: Jestim Espinoza. Tiempo: 2 días.*

7. Conclusiones

A partir del desarrollo del sistema BankTec, se logró dar cumplimiento íntegro a los objetivos específicos planteados para el proyecto:

- **Implementar estructuras tipo registro en memoria y Manejar arreglos estructurados:** Se logró modelar exitosamente la entidad "Cuenta Bancaria" estructurando un bloque contiguo de 28 bytes, anidado dentro de un arreglo estático de

280 bytes (`DB 270 DUP`). Esto demostró que, mediante el direccionamiento indexado (`[BX+SI+offset]`), es posible gestionar colecciones de datos complejos a muy bajo nivel.

- **Diseñar un sistema con múltiples operaciones interdependientes y Utilizar subrutinas (CALL/RET):** Se alcanzó un alto nivel de cohesión y bajo acoplamiento al modularizar el sistema. El uso intensivo de `PROC`, `CALL` y `RET` permitió aislar la lógica de cada operación (creación, depósito, retiro, consultas), facilitando que funciones transaccionales reutilizaran subrutinas utilitarias como la búsqueda lineal sin generar redundancia de código.
- **Aplicar validaciones numéricas y Aplicar lógica condicional avanzada:** Se protegió la integridad de la base de datos del banco aplicando condiciones rigurosas (`CMP`, `JE`, `JA`, `JB`). Se logró implementar exitosamente bloqueos lógicos que impiden violaciones a las reglas de negocio, tales como: creación de cuentas con números duplicados, transacciones sobre cuentas inactivas, ingresos de montos negativos y retiros que provoquen sobregiro.
- **Implementar operaciones aritméticas con números enteros y decimales simulados:** Se logró superar la limitación natural de 16 bits del procesador 8086 mediante la técnica de Punto Fijo. Operando en cascada con `ADD`/`ADC` y `SUB`/`SBB`, se demostró la viabilidad de procesar valores de 32 bits emulando con total precisión matemática hasta cuatro cifras decimales.

8. Recomendaciones

Con base en los objetivos alcanzados y las conclusiones obtenidas, se proponen las siguientes recomendaciones para futuros desarrollos en lenguaje ensamblador:

- **Sobre las estructuras y arreglos:** Se recomienda documentar visualmente el "mapa de memoria" de cada offset diseñado. Dado que el cálculo manual de desplazamientos en arreglos es sumamente estricto, cualquier alteración en el tamaño del registro requiere un recálculo total para evitar la corrupción de datos adyacentes (`buffer overflow`).

9. Bibliografía consultada

- Instituto Tecnológico de Costa Rica. (2026). *Especificaciones de Tarea 1: BankTec*. Paradigmas de Programación (CE1106).
- Rivera Meneses, M. (2026). *Material de apoyo del curso Paradigmas de Programación*. Diapositivas y referencias de cátedra.
- Hyde, R. (1996). *The Art of Assembly Language Programming*. (Capítulos sobre Aritmética Extendida y Precisión Múltiple: Explicación y formulación matemática para el procesamiento algorítmico de operandos de 32 bits dentro de registros nativos de 16 bits mediante el encadenamiento de las banderas de estado y el uso de instrucciones especiales como `ADC` y `SBB`)

8. Enlace al repositorio
https://github.com/JestimEM/TareaASMI2026.git